

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ
федеральное государственное автономное образовательное учреждение
высшего образования «Санкт-Петербургский политехнический
университет Петра Великого»

Институт компьютерных наук и кибербезопасности

Высшая школа технологий искусственного интеллекта

Направление: 02.03.01 «Математика и компьютерные науки»

Лабораторная работа №4
«Доработка компилятора языка Milan»
по дисциплине
«Математическая логика и теория формальных языков»
Вариант 5

Студент,

группы 5130201/30101

_____ Мелешенко С.И.

Преподаватель

_____ Востров А. В.

«_____» _____ 2026г.

Санкт-Петербург, 2026

Содержание

Введение	3
1 Математическое описание	4
1.1 Лексический анализ	4
1.2 Синтаксический анализ	4
1.3 Грамматика рекурсивного спуска	5
1.4 Обзор языка MiLan	5
1.5 Архитектура компилятора	5
1.6 Цикл <i>for</i>	6
1.7 Грамматика языка MiLan	8
1.8 Синтаксическая диаграмма	9
2 Особенности реализации цикла <i>for</i> в компиляторе CMILAN	11
2.1 Новые токены (scanner.h)	11
2.2 Инициализация ключевых слов (scanner.h)	11
2.3 Имена токенов для сообщений об ошибках	12
2.4 Объявление метода синтаксического анализа	13
2.5 Добавление ветки разбора в statement()	13
2.6 Реализация метода forStatement()	14
2.7 Модификация метода factor() для обработки переменных в границах	16
2.8 Вспомогательные методы для работы с переменными	17
3 Результаты работы	19
Заключение	25
Список литературы	26

Введение

Модифицировать компилятор языка Милан (SMILAN) путём добавления новой управляющей конструкции – цикла с параметром `for`. Реализовать оператор цикла, а также проверить программу на для нового оператора.

1	for <переменная> from <выражение> to <выражение>
2	do <список операторов> od

Задачи.

1. Анализ цикла `for` в различных языках программирования.
2. Реализация оператора цикла `for` в компиляторе языка Milan в соответствии с заданным синтаксисом.
3. Генерация объектного кода для виртуальной машины.
4. Проверка программы на тестовых примерах.

1 Математическое описание

Формальный язык — это (любое) множество цепочек, построенных из символов конечного словаря (над конечным словарем). В нем одинаковые сочетания знаков всегда имеют одинаковый смысл.

Транслятор — переводчик с языка программирования на язык двоичных команд.

1.1 Лексический анализ

В реальных трансляторах языков программирования (ЯП) первой фазой является так называемый лексический анализ входной программы — предварительная обработка входного текста с выделением в нем структурно значимых единиц — лексем — минимальных единиц языка, которые имеют смысл.

Лексический анализатор (лексер) — это конечный автомат, который преобразует входную строку символов в последовательность токенов. Формально его можно описать следующим образом:

$$F_{\text{lexer}} : \Sigma^* \rightarrow (T \times D)^*$$

- Σ — входной алфавит (множество допустимых символов)
- T — множество типов токенов
- D — множество допустимых значений токенов

1.2 Синтаксический анализ

Синтаксический анализ — восстановление дерева вывода данной цепочки в данной грамматике.

Существует два типа алгоритмов синтаксического анализа:

- Нисходящие.

Начиная от начального символа дерева пытаемся найти, какой альтернативой заменить очередной нетерминал, чтобы раскрыть новый узел синтаксического дерева. Восстанавливает левый канонический вывод.

- Восходящие.

Начиная от терминальной строки листьев дерева пытаемся найти "связку правую часть продукции грамматики, которую нужно заменить нетерминалом, чтобы получить новый узел синтаксического дерева. Восстанавливает правый канонический вывод.

1.3 Грамматика рекурсивного спуска

LL(k)-грамматики позволяют выполнять нисходящий синтаксический анализ, просматривая входную цепочку слева направо для построения канонического левого вывода. Здесь k указывает, сколько символов из непрочитанной части входной цепочки используется для принятия решений.

Грамматика рекурсивного спуска — это такая LL(1)-грамматика, которая представляется такими синтаксическими диаграммами для каждого нетерминала, что в любом развитии каждой синтаксической диаграммы выбор альтернативного пути может быть сделан на основе анализа одного очередного терминального символа.

1.4 Обзор языка MiLan

Язык Милан — учебный язык программирования.

Программа на Милане представляет собой последовательность операторов, заключенных между ключевыми словами `begin` и `end`. Операторы отделяются друг от друга точкой с запятой. После последнего оператора в блоке точка с запятой не ставится. Компилятор `CMiLan` не учитывает регистр символов в именах переменных и ключевых словах.

В базовую версию языка Милан входят следующие конструкции: константы, идентификаторы, арифметические операции над целыми числами, операторы чтения чисел со стандартного ввода и печати чисел на стандартный вывод, оператор присваивания, условный оператор, оператор цикла с предусловием `while`.

1.5 Архитектура компилятора

Компилятор `CMiLan` включает три компонента:

1. лексический анализатор;
2. синтаксический анализатор;
3. генератор команд виртуальной машины Милана.

Лексический анализатор посимвольно читает из входного потока текст программы и преобразует группы символов в лексемы (терминальные символы грамматики языка Милан). При этом он отслеживает номер текущей строки, который используется синтаксическим анализатором при формировании сообщений об ошибках. Также лексический анализатор удаляет пробельные символы и комментарии. При формировании лексем анализатор идентифицирует ключевые слова и последовательности символов (например, оператор присваивания), а также определяет значения числовых констант и имена переменных. Эти значения становятся значениями атрибутов, связанных с лексемами.

Синтаксический анализатор читает сформированную лексическим анализатором последовательность лексем и проверяет ее соответствие грамматике языка Милан. Для этого используется метод рекурсивного спуска. Если в процессе синтаксического анализа обнаруживается ошибка, анализатор формирует сообщение об ошибке,

включающее сведения об ошибке и номер строки, в которой она возникла. В процессе анализа программы синтаксический анализатор генерирует набор машинных команд, соответствующие каждой конструкции языка.

Генератор кода представляет собой служебный компонент, ответственный за формирование внешнего представления генерируемого кода. Генератор поддерживает буфер команд и предоставляет синтаксическому анализатору набор функций, позволяющий записать указанную команду по определенному адресу. После того, как синтаксический анализатор заканчивает формирование программы, генератор кода используется для печати на стандартный вывод отсортированной по возрастанию адресов последовательности инструкций. Использование генератора кода несколько упрощает устройство компилятора, поскольку синтаксический анализатор может генерировать команды не в порядке их следования в программе, а по мере получения всей необходимой информации для их формирования. Это особенно важно при трансляции таких инструкций, как условные операторы или циклы.

Все три компонента объединяются «драйвером» — управляющей программой компилятора. Драйвер анализирует аргументы командной строки, инициализирует синтаксический анализатор и вызывает его метод `parse()`, запуская процесс трансляции.

SMILAN является примером простейшего однопроходного компилятора, в котором синтаксический анализ и генерация кода выполняются совместно. При этом компилятор никак не оптимизирует код. Реальные компиляторы обычно выполняют трансляцию и оптимизацию кода в несколько проходов, используя несколько видов внутреннего представления программы. Сведения об архитектуре таких компиляторов можно найти, в частности, в классической «книге дракона» [7]. В случае SMILAN, тем не менее, предпочтение было отдано не качеству генерируемого кода, а простоте реализации и легкости расширения.

1.6 Цикл *for*

Циклы — это основные управляющие конструкции в программировании.

Цикл `for` в псевдокоде используется для повторения блока действий заданное количество раз, обычно с использованием счетчика. Стандартная структура включает инициализацию, условие продолжения и шаг изменения.

```
1   ДЛЯ i ОТ начальное_значение ДО конечное_значение
2   // Действия (тело цикла)
3 КОНЕЦ ДЛЯ
```

Pascal

Цикл `for` может иметь одну из двух форм:

1. Для формы `to-do` параметр цикла меняется от начального значения до конечного значения, увеличиваясь всякий раз на единицу:

```
1   for переменная := начальное значение to конечное значение do
```

2	оператор
---	----------

2. Для формы to-downto параметр цикл уменьшается на единицу от начального до конечного значения:

1	for переменная := начальное значение downto конечное значени
	е do
2	оператор

- Верхняя граница включается.
- Вложенный цикл «for i ... for i ...» не допускается (переменная цикла локальна для внешнего блока, но во внутреннем цикле нельзя использовать ту же переменную без отдельного объявления).
- Переменные цикла, определяющие начальное и конечное значение, должны быть объявлены заранее.

C++

Цикл for имеет следующее формальное определение:

1	for (инициализатор; условие; итерация)
2	{
3	<i>// тело цикла</i>
4	}

1. Инициализатор выполняется один раз при начале выполнения цикла и представляет установку начальных условий, как правило, это инициализация счетчиков - специальных переменных, которые используются для контроля за циклом.

2. Условие представляет условие, при соблюдении которого выполняется цикл. Как правило, в качестве условия используется операция сравнения, и если она возвращает ненулевое значение (то есть условие истинно), то выполняется тело цикла, а затем выполняется итерация.

3. Итерация выполняется после каждого завершения блока цикла и задает изменение параметров цикла. Обычно здесь происходит увеличение счетчиков цикла.

- Верхняя граница включается, если используется $\leq$.
- Вложенный цикл «for i ... for i ...» допускается.
- Компилятор может предупредить об использовании неинициализированной переменной, задающей границы.

Java

For цикл используется следующим образом:

1	for (<начальная точка>; <условие выхода>; <операторы счетчика
	>) {
2	<i>// Тело цикла</i>

3 }

- Верхняя граница включается, если используется $\leq$.
- Вложенный цикл «for i ... for i ...» допускается.
- Компилятор проверяет, что локальные переменные инициализированы перед первым использованием. Если вы попытаетесь использовать неинициализированную переменную a или b в конструкции «for (int i = a; i < b; i++)», программа не скомпилируется.

Python

В Python цикл for работает с итерируемыми объектами (списки, строки, кортежи, range и др.):

```
1  for <переменная> in <итерируемый_объект>:  
2  # Тело цикла  
3  <действия>
```

- Верхняя граница не включается.
- Вложенный цикл «for i ... for i ...» допускается, но внешняя переменная i будет перезаписана внутренним циклом.
- Если переменная не определена и её использовать в цикле, будет вызвано исключение NameError во время выполнения программы.

1.7 Грамматика языка MiLan

Грамматика языка Милан, расширенная оператором цикла for, в форме БНФ приведена ниже. Добавленные продукции выделены зелёным цветом.

```
<program> ::= 'begin' <statementList> 'end'  
<statementList> ::= <statement> ';' <statementList>  
                | epsilon  
<statement> ::= <ident> ':' <expression>  
                | 'if' <relation> 'then' <statementList> ['else' <statementList>] 'fi'  
                | 'while' <relation> 'do' <statementList> 'od'  
                | 'write' '(' <expression> ')'  
                | 'for' <ident> 'from' <expression> 'to' <expression> 'do' <statementList>  
'od'  
<expression> ::= <term> <addop> <term>  
<term> ::= <factor> <mulop> <factor>  
<factor> ::= <ident> | <number> | '(' <expression> ')' | 'read' | '-' <factor>  
<relation> ::= <expression> <cmp> <expression>
```


$\langle \text{addop} \rangle ::= '+' | '-'$
 $\langle \text{mulop} \rangle ::= '*' | '/'$
 $\langle \text{cmp} \rangle ::= '=' | '!=' | '<' | '<=' | '>' | '>='$
 $\langle \text{number} \rangle ::= \langle \text{digit} \rangle \langle \text{digit} \rangle$
 $\langle \text{ident} \rangle ::= \langle \text{letter} \rangle \langle \text{letter} \rangle | \langle \text{digit} \rangle$
 $\langle \text{letter} \rangle ::= 'a' | 'b' | 'c' | \dots | 'z' | 'A' | 'B' | 'C' | \dots | 'Z'$
 $\langle \text{digit} \rangle ::= '0' | '1' | '2' | '3' | '4' | '5' | '6' | '7' | '8' | '9'$

Нетерминалы указаны в угловых скобках, символ $::=$ используется вместо стрелки, альтернативы одного и того же терминала записываются через вертикальную черту.

Грамматика языка Милан — контекстно-свободная (КС-грамматика).

Milan является грамматикой LL(1) с рекурсивным спуском, так как необходимо просмотреть поток всего на один символ вперед и все правила начинаются с разных терминальных символов и отсутствует левая рекурсия.

1.8 Синтаксическая диаграмма

Синтаксическая диаграмма приведена на Рис. 1.

Используются сокращения:

Prog — программа,

L — список операторов (statementList),

S — оператор (statement),

E — выражение (expression),

T — терм (term),

P — множитель (factor),

B — условие (relation),

i_k — идентификатор,

c_k — константа (число),

q_k — оператор сравнения,

$addop_k$ — аддитивная операция (+ или -),

$mulop_k$ — мультипликативная операция (* или /).

y_0 : C:=0; Обнуление счётчика команд (или инициализация начала генерации кода). y_20 : Gen(C: STOP)

y_1 : Gen(C: LD Nc+k); C++; Загрузка константы (Nc+k) в стек.

y_2 : Gen(C: LD k); C++; Загрузка переменной с адресом k в стек.

y_3 : Gen(C: IN); C++; Операция ввода (read) — читает число со стандартного ввода и помещает его на стек.

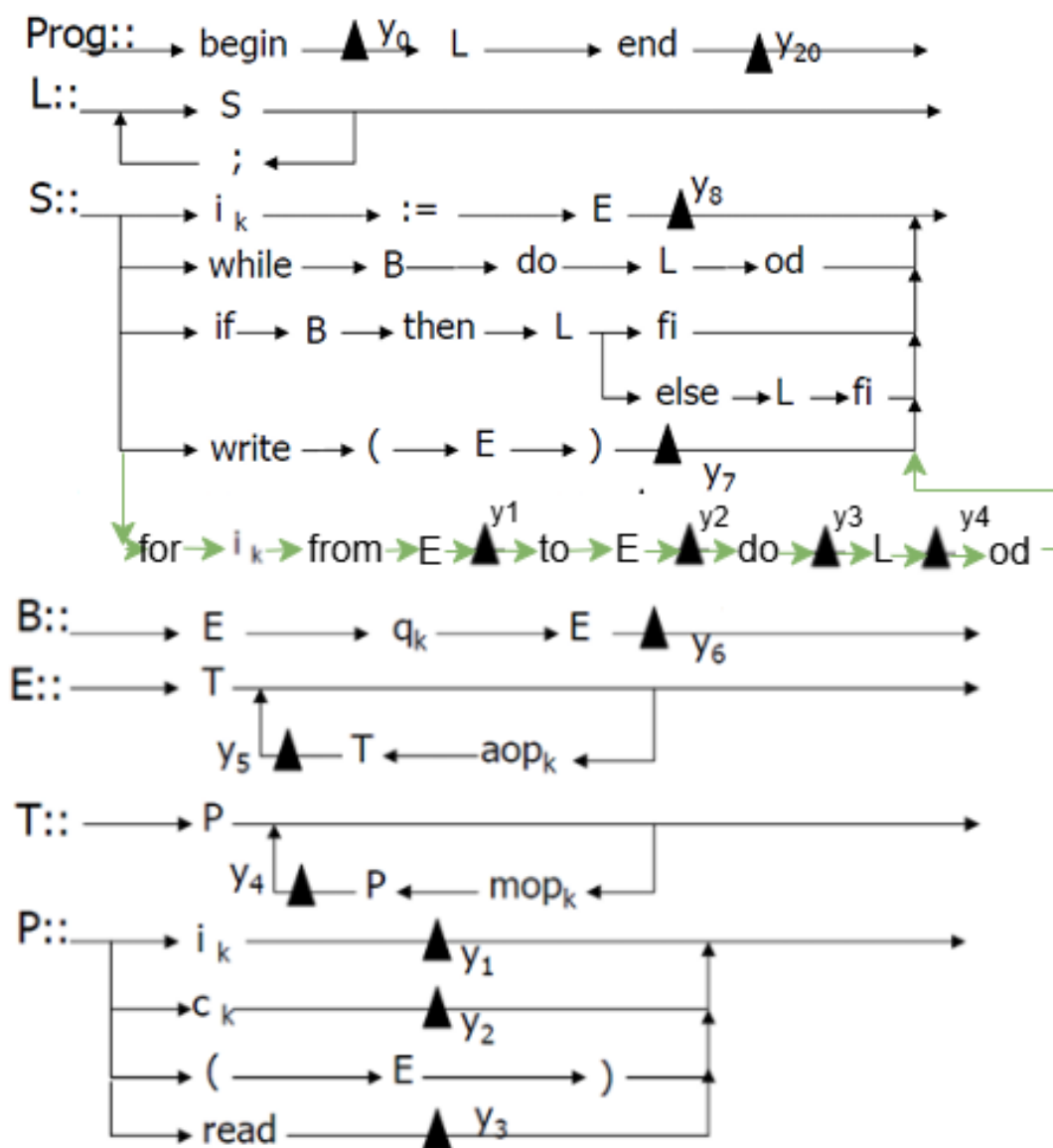


Рис. 1. Синтаксическая диаграмма.

y_4 : $k=0?$ Gen(C: MUL) : Gen(C: DIV) C++; Генерация умножения (MUL) при $k=0$ или деления (DIV) иначе.

y_5 : Gen(C: CMP k); C++; Генерация сравнения с кодом k.

y_6 : Gen(C: OUT); C++; Вывод значения с вершины стека (PRINT).

y_7 : Gen(C: ST Nc+k); C++; Сохранение значения с вершины стека в память по адресу Nc+k.

2 Особенности реализации цикла for в компиляторе SMILAN

2.1 Новые токены (scanner.h)

В перечисление Token были добавлены три новых элемента, соответствующие ключевым словам for, from и to. Это необходимо для их распознавания лексическим анализатором.

Листинг 1. Добавление токенов в enum Token (файл scanner.h)

```
1 enum Token {
2     T_EOF,           // Конец текстового потока
3     T_ILLEGAL,       // Признак недопустимого символа
4     T_IDENTIFIER,    // Идентификатор
5     T_NUMBER,        // Целочисленный литерал
6     T_BEGIN,         // Ключевое слово "begin"
7     T_END,           // Ключевое слово "end"
8     T_IF,            // Ключевое слово "if"
9     T_THEN,          // Ключевое слово "then"
10    T_ELSE,           // Ключевое слово "else"
11    T_FI,             // Ключевое слово "fi"
12    T_WHILE,          // Ключевое слово "while"
13    T_DO,             // Ключевое слово "do"
14    T_OD,             // Ключевое слово "od"
15    T_WRITE,          // Ключевое слово "write"
16    T_READ,           // Ключевое слово "read"
17    T_ASSIGN,         // Оператор "=="
18    T_ADDOP,          // Сводная лексема для "+" и "-" (операция типа сл
                          // ожения)
19    T_MULOP,          // Сводная лексема для "*" и "/" (операция типа ум
                          // ножения)
20    T_CMP,            // Сводная лексема для операторов отношения
21    T_LPAREN,         // Открывающая скобка
22    T_RPAREN,         // Закрывающая скобка
23    T_SEMICOLON,      // ";"
24    T_FOR,            // новое
25    T_FROM,           // новое
26    T_TO,             // новое
27 };
```

2.2 Инициализация ключевых слов (scanner.h)

В конструкторе класса Scanner добавлены записи в ассоциативный массив keywords_, связывающие строки "for" "from" "to" с соответствующими токенами. Регистр символов при этом игнорируется (все ключевые слова приводятся к нижнему

регистру).

Листинг 2. Добавление ключевых слов (файл scanner.h)

```
1  explicit Scanner(const string& fileName, istream& input)
2      : fileName_(fileName), lineNumber_(1), input_(input)
3  {
4      keywords_["begin"] = T_BEGIN;
5      keywords_["end"] = T_END;
6      keywords_["if"] = T_IF;
7      keywords_["then"] = T_THEN;
8      keywords_["else"] = T_ELSE;
9      keywords_["fi"] = T_FI;
10     keywords_["while"] = T_WHILE;
11     keywords_["do"] = T_DO;
12     keywords_["od"] = T_OD;
13     keywords_["write"] = T_WRITE;
14     keywords_["read"] = T_READ;
15     keywords_["for"] = T_FOR;
16     keywords_["from"] = T_FROM;
17     keywords_["to"] = T_TO;
18
19     nextChar();
20 }
21
```

2.3 Имена токенов для сообщений об ошибках

В массиве tokenNames_ добавлены строковые описания новых токенов, чтобы при возникновении ошибки компилятор выводил понятное сообщение.

Листинг 3. Обновление массива tokenNames

```
1  static const char * tokenNames_[] = {
2      "end of file",
3      "illegal token",
4      "identifier",
5      "number",
6      "'BEGIN'",
7      "'END'",
8      "'IF'",
9      "'THEN'",
10     "'ELSE'",
11     "'FI'",
12     "'WHILE'",
13     "'DO'",
14     "'OD'",
15     "'WRITE'",
16     "'READ'",

```

```

17  "':=' ",
18  "'+' or '-' ",
19  "'*' or '/' ",
20  "comparison operator",
21  "'(' ",
22  "')' ",
23  "':' ",
24  "'FOR' ",
25  "'FROM' ",
26  "'TO' "
27  };

```

2.4 Объявление метода синтаксического анализа

В классе Parser объявлен метод forStatement() и добавлены вспомогательные поля/методы для проверки переменных в границах на строчку 117 файла parser.h.

Листинг 4. parser.h

```

1 void forStatement();
2 int findVariable(const string& var);
3 int addVariable(const string& var);
4 bool inForBounds_; // флаг: находимся ли внутри границ цикла

```

2.5 Добавление ветки разбора в statement()

В методе statement() добавлена ветка, которая срабатывает при обнаружении токена T_FOR. Она вызывает forStatement() для дальнейшего разбора.

Листинг 5. Ветка в statement()

```

1 void Parser::statement()
2 {
3     if(see(T_IDENTIFIER)) {
4         string name = scanner_>getStringValue();
5         int varAddress = findOrAddVariable(scanner_>getStringValue());
6         next();
7         mustBe(T_ASSIGN);
8         expression();
9         codegen_>emit(STORE, varAddress);
10    }
11    else if(match(T_IF)) {
12        relation();
13        int jumpNoAddress = codegen_>reserve();
14        mustBe(T_THEN);
15        statementList();

```

```

16     if (match(T_ELSE)) {
17         int jumpAddress = codegen_>reserve();
18         codegen_>emitAt(jumpNoAddress, JUMP_NO, codegen_>
19             getCurrentAddress());
20         statementList();
21         codegen_>emitAt(jumpAddress, JUMP, codegen_>
22             getCurrentAddress());
23     }
24     else {
25         codegen_>emitAt(jumpNoAddress, JUMP_NO, codegen_>
26             getCurrentAddress());
27     }
28     mustBe(T_FI);
29 }
30 else if (match(T_WHILE)) {
31     int conditionAddress = codegen_>getCurrentAddress();
32     relation();
33     int jumpNoAddress = codegen_>reserve();
34     mustBe(T_DO);
35     statementList();
36     mustBe(T_OD);
37     codegen_>emit(JUMP, conditionAddress);
38     codegen_>emitAt(jumpNoAddress, JUMP_NO, codegen_>
39         getCurrentAddress());
40 }
41 else if (match(T_FOR)) {
42     forStatement();
43 }
44 else if (match(T_WRITE)) {
45     mustBe(T_LPAREN);
46     expression();
47     mustBe(T_RPAREN);
48     codegen_>emit(PRINT);
49 }
50 else {
51     reportError("statement expected.");
52 }
53 }

```

2.6 Реализация метода forStatement()

Метод выполняет полный разбор и генерацию кода для цикла.

Вход: текущий токен – T_FOR, последовательность (идентификатор, FROM, выражение, TO, выражение, DO, список операторов, OD).

Выход: добавляются инструкции для инициализации, проверки условия, тела, инкремента и переходов.

Сначала выполняется проверка идентификатора – переменная цикла должна следовать за `for`, затем проверка на повторное объявление – если переменная уже существует, выводится ошибка (`loop variable already declared`), иначе создаётся новая.

Обработка `FROM` – устанавливается флаг `inForBounds_ = true`, разбирается выражение начального значения, флаг снимается, результат сохраняется в переменную (`STORE`).

Обработка `TO` – запоминается адрес начала проверки условия, генерируется загрузка переменной (`LOAD`), затем разбирается выражение конечного значения, генерируется сравнение `COMPARE 4` (код 4 означает `<=`).

Установлен `JUMP_NO` на случай невыполнения условия, после ключевого слова `DO` разбирается список операторов (`statementList()`), затем проверяется `OD`.

Далее происходит загрузка переменной, добавление 1, сохранение, происходит безусловный переход на адрес проверки условия.

Заполнение зарезервированного перехода – адрес выхода из цикла.

Листинг 6. Ветка в `statement()`

```
1 void Parser::forStatement() {
2     if (!see(T_IDENTIFIER)) {
3         reportError("identifier expected after for");
4         recover(T_DO);
5         return;
6     }
7     string varName = scanner_>getStringValue();
8     // проверка на повторное объявление
9     if (findVariable(varName) != -1) {
10        reportError("loop variable '" + varName + "' already
11            declared");
12        recover(T_DO);
13        return;
14    }
15    int varAddr = addVariable(varName);    // создаём новую
16    next();
17
18    mustBe(T_FROM);
19    inForBounds_ = true;
20    expression();
21    inForBounds_ = false;
22    codegen_>emit(STORE, varAddr);
23
24    mustBe(T_TO);
25    int conditionAddr = codegen_>getCurrentAddress();
26    codegen_>emit(LOAD, varAddr);
27    inForBounds_ = true;
28    expression();
29    inForBounds_ = false;
30    codegen_>emit(COMPARE, 4);
```

```

30
31     int jumpNoAddr = codegen_>reserve();
32     mustBe(T_DO);
33     statementList();
34     mustBe(T_OD);
35
36     codegen_>emit(LOAD, varAddr);
37     codegen_>emit(PUSH, 1);
38     codegen_>emit(ADD);
39     codegen_>emit(STORE, varAddr);
40     codegen_>emit(JUMP, conditionAddr);
41     codegen_>emitAt(jumpNoAddr, JUMP_NO, codegen_>
        getCurrentAddress());
42 }

```

2.7 Модификация метода factor() для обработки переменных в границах

Чтобы переменные в выражениях from и to не создавались автоматически (а только использовались существующие), добавлен флаг inForBounds_. При его истинности вызывается findVariable() (только поиск), а при ложности – findOrAddVariable() (поиск или создание).

Листинг 7. factor()

```

1 void Parser::factor()
2 {
3     /*
4      * Множитель описывается следующими правилами:
5      * <factor> -> number | identifier | -<factor> | (<expression>)
6      * | READ
7      */
8     if (see(T_NUMBER)) {
9         int value = scanner_>getIntValue();
10        next();
11        codegen_>emit(PUSH, value);
12        //Если встретили число, то преобразуем его в целое и записываем на вершину стека
13    }
14    else if (see(T_IDENTIFIER)) {
15        string name = scanner_>getStringValue();
16        int varAddress;
17        if (inForBounds_) {
18            // В границах цикла только существующие переменные
19            varAddress = findVariable(name);
20            if (varAddress == -1) {
21                reportError("undefined variable '" + name + "'");

```



```

21         next();
22         return;
23     }
24 }
25 else {
26     // В обычных выражениях (присваивание и т.д.) создаём
27     // при необходимости
28     varAddress = findOrAddVariable(name);
29 }
30 next();
31 codegen_ -> emit(Load, varAddress);
32 }
33 else if (see(T_ADDOP) && scanner_ -> getArithmeticValue() ==
34     A_MINUS) {
35     next();
36     factor();
37     codegen_ -> emit(INVERT);
38     //Если встретили знак "-", и за ним <factor> то инвертируем
39     //значение, лежащее на вершине стека
40 }
41 else if (match(T_LPAREN)) {
42     expression();
43     mustBe(T_RPAREN);
44     //Если встретили открывающую скобку, тогда следом может идти
45     //любое арифметическое выражение и обязательно
46     //закрывающая скобка.
47 }
48 else if (match(T_READ)) {
49     codegen_ -> emit(INPUT);
50     //Если встретили зарезервированное слово READ, то записываем
51     //на вершину стека идет запись со стандартного ввода
52 }
53 else {
54     reportError("statement expected.");
55 }
56 }

```

2.8 Вспомогательные методы для работы с переменными

В parser.cpp добавлены два метода.

findVariable(const string& var) - поиск переменной по имени без её создания.

Ищет переданное имя var в массиве variables_. Если переменная найдена, возвращает её адрес (it->second). Иначе возвращает -1.

Вход: var – имя переменной (строка).

Выход: int – адрес переменной в памяти виртуальной машины, если переменная найдена; -1 – если переменная не существует.

Листинг 8. findVariable

```
1 int Parser::findVariable(const string& var) {  
2     VarTable::iterator it = variables_.find(var);  
3     if (it != variables_.end())  
4         return it->second;  
5     return -1;    // не найдена  
6 }
```

addVariable(const string& var) - поиск переменной по имени без её создания.

В таблицу variables_ по ключу var записывается текущее значение lastVar_ (счётчик последнего выделенного адреса). Затем lastVar_ увеличивается на 1. Возвращается адрес, по которому создана переменная.

Вход: var – имя новой переменной (строка).

Выход: int – адрес, выделенный для переменной (текущее значение lastVar_ до инкремента).

Листинг 9. addVariable

```
1 int Parser::addVariable(const string& var) {  
2     variables_[var] = lastVar_ ;  
3     return lastVar_ ++;  
4 }
```

3 Результаты работы

Запуск:

1. VS Сборка: Ctrl+Shift+B.

2. Переход в нужную директорию cmd (файлы .mil, milanvm.exe лежат там):

```
1 cd Downloads\cmilan\cmilan/src
```

3. Компиляция в cmd:

```
1 cmilan_vs2011\Debug\cmilan.exe testfor.mil > testfor.out
```

4. Запуск на виртуальной машине:

```
1 milanvm.exe testfor.out
```

5. Просмотр содержимого testfor.out:

```
1 type testfor.out
```

Программа №1: простой цикл (файл testfor.mil)

Исходный код:

Листинг 10. Простой цикл for

```
1 begin
2   for i from 1 to 5 do
3     write(i)
4   od
5 end
```

Компилятор сгенерировал следующий код (Рис. 2):

```
C:\Users\Sofia\Downloads\cmilan\cmilan\src>type testfor.out
0:      PUSH      1
1:      STORE     0
2:      LOAD      0
3:      PUSH      5
4:      COMPARE   4
5:      JUMP_NO    13
6:      LOAD      0
7:      PRINT
8:      LOAD      0
9:      PUSH      1
10:     ADD
11:     STORE     0
12:     JUMP      2
13:     STOP
```

Рис. 2. Сгенерированный код.

Анализ сгенерированного кода:

- Адреса 0–1: инициализация цикла — установка начального значения $i = 1$ (PUSH 1, STORE 0).
- Адрес 2: загрузка текущего значения переменной i на стек (LOAD 0).
- Адрес 3: загрузка верхней границы 5 на стек (PUSH 5).
- Адрес 4: сравнение $i \leq 5$ (код сравнения 4).
- Адрес 5: условный переход на выход (адрес 13), если условие ложно (JUMP_NO 13).
- Адреса 6–7: тело цикла — печать значения i (LOAD 0, PRINT).
- Адреса 8–11: инкремент i на 1 и сохранение (LOAD 0, PUSH 1, ADD, STORE 0).
- Адрес 12: безусловный переход на проверку условия (адрес 2).
- Адрес 13: остановка (STOP).

Вывод виртуальной машины (Рис. 3):

```
C:\Users\Sofia\Downloads\cmilan\cmilan\src>milanvm.exe testfor.out
Reading input from testfor.out
1
2
3
4
5
```

Рис. 3. Вывод виртуальной машины.

Результат соответствует ожидаемому.

Программа №2: различные тестовые случаи (файл testforMAIN.mil)

В одном файле были последовательно выполнены 13 различных циклов `for`, разделённых точкой с запятой. Исходный код приведён в листинге 11.

Листинг 11. Комплексный тест (фрагменты)

```
1 begin
2   /* 1. Начальное значение больше конечного      тело не выполн
   яется */
3   for i from 5 to 1 do
4     write(i)
5   od;
6   /* 2. Начальное значение равно конечному      одна итерация
   */
7   for a from 3 to 3 do
8     write(a)
9   od;
```

```

10  /* 3. Границы выражения */
11  for b from 1*2 to 2+2 do
12      write(b)
13  od;
14  /* 4. Вложенные циклы */
15  for c from 1 to 4 do
16      for j from 1 to c do
17          write(j)
18      od
19  od;
20  /* 5. Изменение переменной цикла внутри тела (двойной инкремент) */
21  for w from 1 to 6 do
22      write(w);
23      w := w + 1
24  od;
25  /* 6. Отрицательные границы */
26  for i1 from -2 to 2 do
27      write(i1)
28  od;
29  /* 7. Обычный. */
30  for i2 from 1 to 3 do
31      write(i2)
32  od;
33  /* 8. Ничего не делаем, просто прокручиваем */
34  for i3 from 1 to 10 do
35  od;
36  /* 9. Сумма чисел от 1 до 5 */
37  s := 0;
38  for i4 from 1 to 5 do
39      s := s + i4
40  od;
41  write(s);
42  /* 10. Факториал 4! */
43  f := 1;
44  for i5 from 1 to 4 do
45      f := f * i5
46  od;
47  write(f);
48  /* 11. От a1 */
49  a1 := 1;
50  for i6 from a1 to 4 do
51      write(i6)
52  od;
53  /* 12. До b1 */
54  b1 := 7;
55  for i7 from 3 to b1 do
56      write(i7)

```

```

57     od;
58     /* 13. От a1 до b1*/
59     for i8 from a1 to b1 do
60         write(i8)
61     od
62 end

```

После компиляции и запуска на виртуальной машине получен следующий вывод (Рис. 4):

```

C:\Users\Sofia\Downloads\cmilan\cmilan\src>milanvm.exe testfor.out
Reading input from testfor.out
3
2
3
4
1
1
2
1
2
2
3
1
2
3
3
4
1
3
5
-2
-1
0
1
2
1
2
3
15
24
1
2
3
4
3
4
5
6
7
1
2
3
4
5
6
7

```

Рис. 4. Вывод.

1. С 5 до 1 — ничего не вывело (пропуск).
2. С 3 до 3 — вывело 3.
3. От 2 до 4 — вывело 2 3 4.

4. Вложенные циклы — вывело треугольник: 1, 1 2, 1 2 3, 1 2 3 4.
 5. Двойной инкремент — шаг 2: 1 3 5.
 6. От -2 до 2 — -2 -1 0 1 2.
 7. От 1 до 3 — 1 2 3.
 8. Пустой цикл — ничего не вывел.
 9. Сумма — вывело 15.
 10. Факториал — вывело 24.
 11. От переменной a1 — вывело 1, 2, 3, 4.
 12. До переменной b1 — вывело 3, 4, 5, 6, 7.
 13. От переменной a1 до переменной b1 — вывело 1, 2, 3, 4, 5, 6, 7.
- Все результаты соответствуют ожиданиям.

Программа №3: обработка синтаксических ошибок

Для проверки были созданы программы с намеренными ошибками. В таблице 1 приведены основные ошибки.

Таблица 1

Синтаксические ошибки цикла for и сообщения компилятора

Ошибочная конструкция	Сообщение компилятора
for i from 1 to 5 write(i) od	Line 3: 'WRITE' found while 'DO' expected.
for i from 1 to 5 do write(i) end	Line 4: 'END' found while 'OD' expected.
for i from 1 to 5 do; write(i) od	Line 2: statement expected.
for i from 1 to 5 do write(i) od; end	Line 5: statement expected.
for from 1 to 5 do write(i) od	Line 2: identifier expected after for
for i 1 to 5 do write(i) od	Line 2: identifier expected after for
for i from 1 5 do write(i) od	Line 2: number found while 'TO' expected.
for i to 5 from 1 do write(i) od	Line 2: identifier expected after for
for i from to 5 do write(i) od	Line 2: statement expected.

продолжение на следующей странице

Ошибочная конструкция	Сообщение компилятора
for i from 1 to do write(i) od	Line 2: statement expected.
for i from 1 to 5 do ; od	Line 2: statement expected.
for i from 1 to 3 do for j from 1 to i do write(j) od end	Line 4: 'END' found while 'OD' expected.
for i from 1 to 5 write(i); write(i) od	Line 3: 'WRITE' found while 'DO' expected.
for read from 1 to 5 do write(read) od	Line 2: identifier expected after for

Эти тесты подтверждают, что компилятор корректно обнаруживает ошибки в структуре цикла `for`.

Пример вывода (Рис. 5):

```
C:\Users\Sofia\Downloads\cmilan\cmilan\src>cmilan_vs2011\Debug\cmilan.exe testforwithoutexpressio
nfrom.mil > testfor.out
Line 2: statement expected.
```

Рис. 5. Вывод.

Заключение

В ходе выполнения лабораторной работы был успешно расширен компилятор языка Милан путём добавления цикла `for`. Были проанализированы циклы `for` в различных языках программирования (Pascal, C++, Java, Python), реализован оператор цикла `for` в компиляторе языка Милан в соответствии с заданным синтаксисом, выполнена задача генерации объектного кода для виртуальной машины, программа проверена на тестовых примерах.

Грамматика языка Милан осталась контекстно-свободной и LL(1). Синтаксический анализатор, построенный методом рекурсивного спуска, позволил добавить цикл `for` без изменения общей архитектуры.

К достоинствам реализации можно отнести минимальные изменения в существующей архитектуре компилятора — добавлена только одна новая конструкция без нарушения работы остальных операторов, код использует стандартные инструкции виртуальной машины (PUSH, STORE, LOAD, COMPARE, JUMP_NO, JUMP, ADD) и не требует модификации самой ВМ. Также реализована семантическая проверка: переменная цикла не должна быть объявлена ранее, а переменные, используемые в границах `from` и `to`, обязаны быть инициализированы до входа в цикл.

К недостаткам можно отнести отсутствие оптимизации, например, при использовании константных границ можно было бы избежать лишних LOAD и PUSH в цикле - константа (верхняя граница) загружается на стек в каждой итерации.

Программа может быть масштабирована добавлением шага, отличного от 1, добавлением конструкций `continue` и `break` для досрочного выхода из цикла.

Список использованной литературы

- [1] Востров, А.В. Курс лекций по дисциплине «Математическая логика и теория формальных языков». URL <https://tema.spbstu.ru/mathem/> (дата обращения 20.04.2026 г.)
- [2] Типы циклов в языках программирования. URL <https://tproger.ru/explain/typy-ciklov-v-jazykah-programmirovaniya-for-foreach-while-i-do-while> (дата обращения 20.04.2026 г.)
- [3] Цикл for в Python. URL <https://javarush.com/groups/posts/cikl-for-v-python> (дата обращения 20.04.2026 г.)
- [4] Цикл For в Java. URL <https://javarush.com/groups/posts/for-v-java> (дата обращения 20.04.2026 г.)
- [5] Циклы. URL <https://metanit.com/cpp/tutorial/2.13.php> (дата обращения 20.04.2026 г.)
- [6] Оператор цикла for в Pascal. URL https://foxford.ru/wiki/informatika/operator-tsikla-for-v-pascal?srsltid=AfmBOoq1g3VhhXYrHrXgg6bwVBnpSW_s4uvlm2K5juCwn1VusLp_6iK9 (дата обращения 20.04.2026 г.)
- [7] А. Ахо, М. Лам, Р. Сети, Дж. Ульман, *Компиляторы: принципы, технологии и инструментарий*, 2-е изд. М.: Вильямс, 2011.
- [8] Ю.Г. Карпов, *Теория и технология программирования. Основы построения трансляторов*. СПб.: БХВ-Петербург, 2005.